

Manuel technique — Bot WhatsApp Pièces

Plateforme : Pièces — Marketplace de pièces auto d'occasion **Marché :** Côte d'Ivoire (Abidjan) **Devise :** FCFA **Version :** 1.0 — Mars 2026

Table des matières

- [1. Vue d'ensemble du bot WhatsApp](#)
- [2. Architecture technique](#)
- [3. Intégration Meta Cloud API](#)
- [4. Configuration et variables d'environnement](#)
- [5. Webhook : vérification \(GET\)](#)
- [6. Webhook : réception des messages \(POST\)](#)
- [7. Sécurité HMAC SHA-256](#)
- [8. Commandes du bot](#)
- [9. Traitement des images](#)
- [10. Envoi de messages texte](#)
- [11. Templates WhatsApp Meta](#)
- [12. Template : order_confirmation](#)
- [13. Template : delivery_in_transit](#)
- [14. Template : delivery_completed](#)
- [15. Template : owner_options](#)
- [16. Template : artci_consent](#)
- [17. Système de notifications](#)
- [18. Notifications par événement](#)
- [19. Architecture multi-canal](#)
- [20. Préférences de notification utilisateur](#)
- [21. Format des numéros de téléphone](#)
- [22. Intégration avec la machine à états commande](#)
- [23. Intégration avec la machine à états livraison](#)
- [24. Rate limiting et gestion d'erreurs](#)
- [25. Tests unitaires](#)
- [26. État de l'implémentation](#)
- [27. Évolutions planifiées](#)

- 28. Processus de soumission des templates Meta
- 29. Référence des fichiers
- 30. FAQ technique

1. Vue d'ensemble du bot WhatsApp

Le bot WhatsApp de Pièces est un composant central de la plateforme, conçu pour le marché ivoirien où **WhatsApp est le canal de communication dominant**. Il remplit deux fonctions :

1.1 Bot conversationnel (entrant)

Reçoit les messages des utilisateurs et répond :

UTILISATEUR	BOT PIÈCES
"aide" →	"Bienvenue sur Pièces! Envoyez une photo..."
"recherche alternateur" →	"Recherche alternateur – consultez pieces.ci/browse "
[Photo d'une pièce] →	"Photo reçue! Identification en cours..."

1.2 Système de notifications (sortant)

Envoie des notifications proactives aux utilisateurs :

ÉVÉNEMENT SYSTÈME	MESSAGE WHATSAPP
Commande payée →	"Commande confirmée ✅ Pièce: Alternateur..."
Livraison en route →	"Votre pièce est en route 🚀 Coursier Konan..."
Pièce livrée →	"Pièce livrée ✅ La pièce est-elle conforme?"

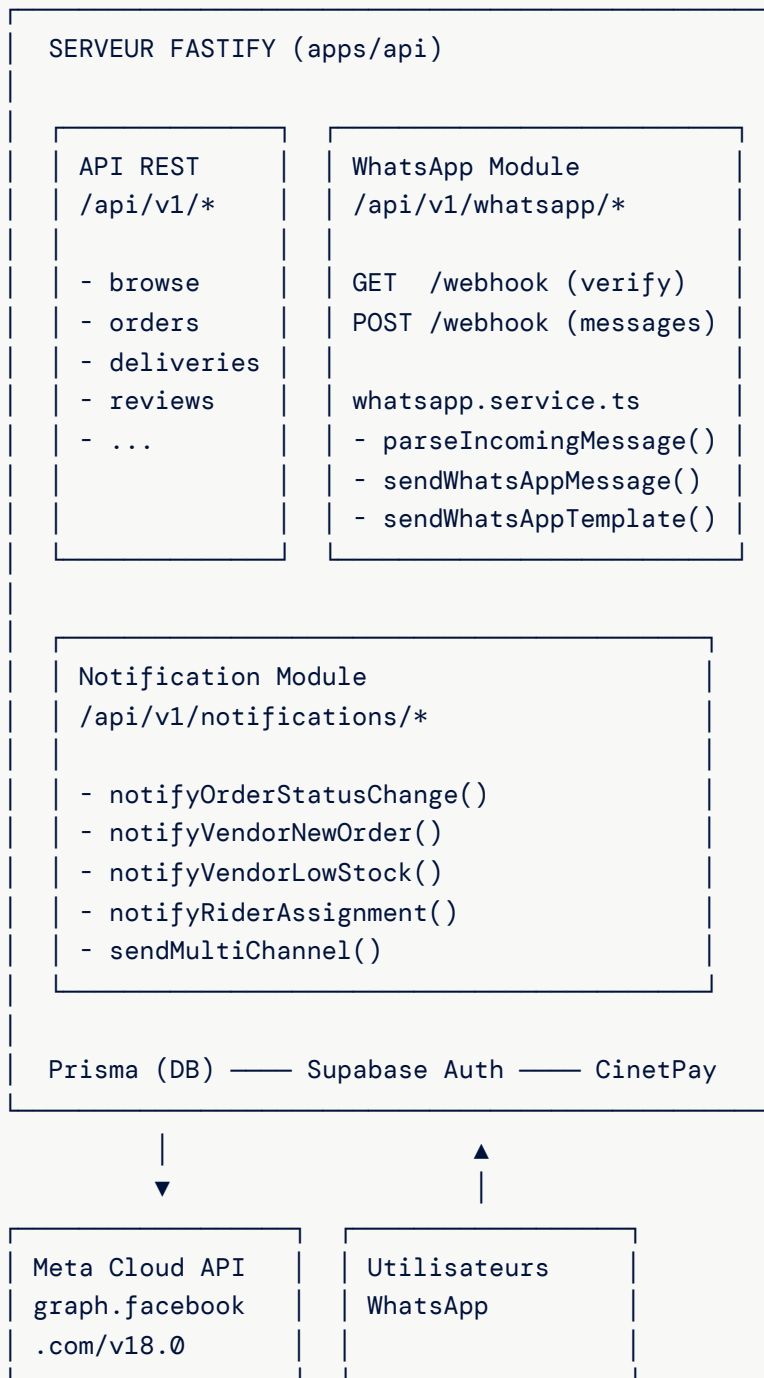
1.3 Pourquoi WhatsApp ?

Critère	Justification
Pénétration	WhatsApp est utilisé par >90% des smartphones en Côte d'Ivoire
Accessibilité	Fonctionne sur connexions 2G/3G lentes
Familiarité	Les utilisateurs connaissent déjà l'interface
Gratuité	Pas de frais SMS pour l'utilisateur
Rich media	Support des images pour l'identification IA

2. Architecture technique

2.1 Architecture monolithique

Le bot WhatsApp s'exécute dans le **même processus Fastify** que l'API REST. Pas de service séparé.



2.2 Flux de données

Message entrant :

Utilisateur → WhatsApp → Meta Cloud API → Webhook POST → Bot → Réponse → Meta → Ut

Notification sortante :

Événement système → Notification Service → sendWhatsAppMessage/Template → Meta → U

3. Intégration Meta Cloud API

3.1 API Graph Facebook

Paramètre	Valeur
Version API	v18.0
URL de base	<code>https://graph.facebook.com/v18.0</code>
Authentification	Bearer Token
Type de contenu	application/json

3.2 Endpoint d'envoi

```
POST https://graph.facebook.com/v18.0/{WHATSAPP_PHONE_ID}/messages
Headers:
  Authorization: Bearer {WHATSAPP_TOKEN}
  Content-Type: application/json
```

3.3 Structure des messages sortants

Message texte simple :

```
{
  "messaging_product": "whatsapp",
  "to": "225070000000000",
  "type": "text",
  "text": {
    "body": "Votre message ici"
  }
}
```

Message template :

```
{
  "messaging_product": "whatsapp",
  "to": "2250700000000",
  "type": "template",
  "template": {
    "name": "order_confirmation",
    "language": { "code": "fr" },
    "components": [{
      "type": "body",
      "parameters": [
        { "type": "text", "text": "Alternateur Toyota" },
        { "type": "text", "text": "Auto Parts Abidjan" },
        { "type": "text", "text": "45000" },
        { "type": "text", "text": "J+2" }
      ]
    }]
  }
}
```

3.4 Limite Meta

Élément	Limite
Messages par seconde	80 (par compte business)
Taille message texte	4096 caractères
Templates par compte	250 (par langue)

4. Configuration et variables d'environnement

4.1 Variables requises

```
# WhatsApp Business API (Meta Cloud API)
WHATSAPP_TOKEN=EAAxxxxxxxxx # Bearer token d'authentification
WHATSAPP_PHONE_ID=123456789 # ID du numéro business
WHATSAPP_VERIFY_TOKEN=pieces-verify-token # Token de vérification webhook
WHATSAPP_APP_SECRET=abcdef123456 # Secret HMAC pour validation webhook
```

4.2 Comportement sans configuration

Variable manquante	Comportement
WHATSAPP_TOKEN	Messages non envoyés, { success: false }
WHATSAPP_PHONE_ID	Messages non envoyés, { success: false }
WHATSAPP_VERIFY_TOKEN	Valeur par défaut : pieces-verify-token
WHATSAPP_APP_SECRET	Vérification HMAC désactivée (dev uniquement)

Le bot démarre sans erreur même si les variables ne sont pas configurées (**dégradation gracieuse**).

5. Webhook : vérification (GET)

5.1 Endpoint

```
GET /api/v1/whatsapp/webhook
```

5.2 But

Meta envoie une requête GET pour **vérifier** que votre serveur est bien le destinataire du webhook. Cette vérification se fait une seule fois lors de la configuration.

5.3 Paramètres reçus

Paramètre	Description
hub.mode	Doit être subscribe
hub.verify_token	Doit correspondre à WHATSAPP_VERIFY_TOKEN
hub.challenge	Chaîne à renvoyer telle quelle

5.4 Flux de vérification

```
Meta Cloud API → GET /webhook?hub.mode=subscribe
                  &hub.verify_token=pieces-verify-token
                  &hub.challenge=abc123
```

Serveur vérifie :

```
hub.mode === 'subscribe' ? 
```

```
hub.verify_token === env var ? 
```

→ Réponse : 200 OK, body: "abc123"

Si échec :

→ Réponse : 403 Forbidden

6. Webhook : réception des messages (POST)

6.1 Endpoint

```
POST /api/v1/whatsapp/webhook
```

6.2 Structure du payload Meta

```
{
  "object": "whatsapp_business_account",
  "entry": [{
    "id": "BUSINESS_ACCOUNT_ID",
    "changes": [{
      "value": {
        "messaging_product": "whatsapp",
        "metadata": {
          "display_phone_number": "225XXXXXXXXXX",
          "phone_number_id": "PHONE_NUMBER_ID"
        },
        "contacts": [{
          "profile": { "name": "Konan Yao" },
          "wa_id": "22507000000000"
        }],
        "messages": [{
          "from": "22507000000000",
          "id": "wamid.xxxxx",
          "timestamp": "1709312400",
          "type": "text",
          "text": { "body": "aide" }
        }]
      },
      "field": "messages"
    }]
  }]
}
```

6.3 Extraction des données

La fonction `parseIncomingMessage()` extrait :

```
{
  from: string    // "22507000000000" - numéro de l'expéditeur
  text: string    // "aide" - contenu du message texte
  imageId: string // ID de l'image (si type: "image")
}
```

Chemin d'extraction :

```
entry[0].changes[0].value.messages[0].from    → numéro
entry[0].changes[0].value.messages[0].text.body → texte
entry[0].changes[0].value.messages[0].image.id → image
```

6.4 Réponse au webhook

Le serveur **retourne toujours 200 OK** pour acquitter la réception. Cela empêche Meta de renvoyer le même message (retry).

7. Sécurité HMAC SHA-256

7.1 Principe

Chaque requête POST du webhook est signée par Meta avec un **HMAC SHA-256**. Cela garantit que le message provient bien de Meta et n'a pas été altéré.

7.2 Processus de vérification

Meta envoie :

Header: x-hub-signature-256 = sha256=abcdef123456...

Body: {"object":"whatsapp_business_account",...}

Serveur vérifie :

1. Calcule HMAC-SHA256(body_brut, WHATSAPP_APP_SECRET)
2. Compare avec la signature reçue
3. Si match → traite le message
4. Si mismatch → 401 Unauthorized

7.3 Fonction de vérification

```
function verifyWebhookSignature(rawBody: string, signature: string): boolean {
  const expectedSig = crypto
    .createHmac('sha256', process.env.WHATSAPP_APP_SECRET)
    .update(rawBody)
    .digest('hex');

  return signature === 'sha256=' + expectedSig;
}
```

7.4 Mode développement

Si `WHATSAPP_APP_SECRET` n'est **pas configuré**, la vérification HMAC est **désactivée**. Cela facilite le développement local mais **ne doit jamais être utilisé en production**.

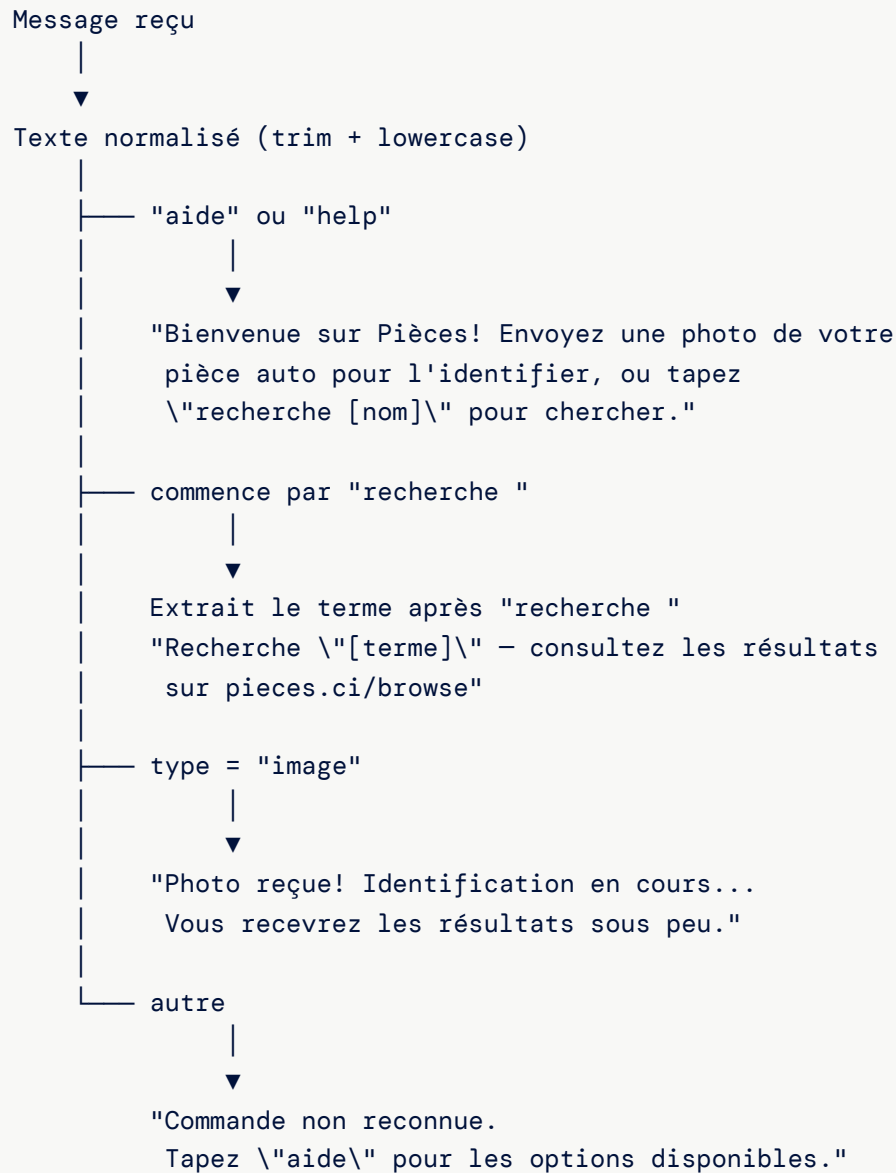
8. Commandes du bot

8.1 Commandes reconnues

Le bot utilise un parseur simple basé sur la correspondance de chaînes (pas de NLP).

Commande	Variantes	Réponse
<code>aide</code>	<code>aide</code> , <code>help</code> (insensible à la casse)	Message d'aide avec les options disponibles
<code>recherche [terme]</code>	<code>recherche alternateur</code> , <code>recherche filtre huile</code>	Redirection vers le catalogue web
[Image]	Tout message de type <code>image</code>	Accusé de réception + notification d'identification
Autre	Tout texte non reconnu	Message d'erreur avec suggestion "aide"

8.2 Flux de traitement



8.3 Caractéristiques

Propriété	Valeur
Sensibilité casse	Insensible (tout converti en minuscules)
Historique conversation	Non (bot sans état)
NLP / IA	Non (correspondance simple de chaînes)
Multi-tour	Non (chaque message traité indépendamment)

9. Traitement des images

9.1 Réception d'image

Quand un utilisateur envoie une photo de pièce :

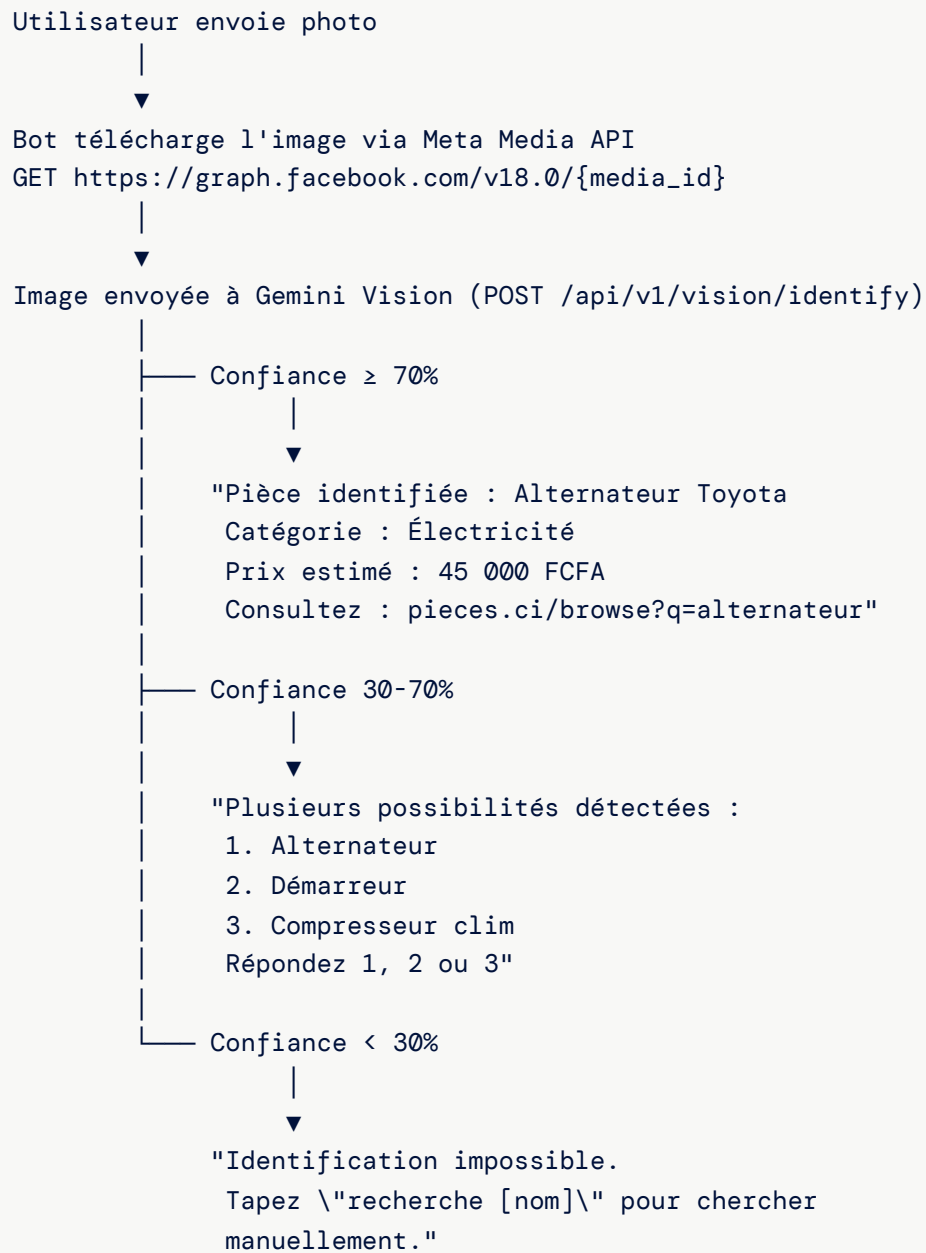
```
{  
  "type": "image",  
  "image": {  
    "id": "media_id_from_meta",  
    "mime_type": "image/jpeg",  
    "sha256": "..."  
  }  
}
```

9.2 Réponse actuelle (v1)

Le bot envoie un accusé de réception :

```
"Photo reçue! Identification en cours...  
Vous recevrez les résultats sous peu."
```

9.3 Flux prévu (Phase 2)



10. Envoi de messages texte

10.1 Fonction d'envoi

```
async function sendWhatsAppMessage(to: string, message: string):
    Promise<SendResult> {
    const response = await fetch(
        `https://graph.facebook.com/v18.0/${WHATSAPP_PHONE_ID}/messages`,
        {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${WHATSAPP_TOKEN}`,
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({
                messaging_product: 'whatsapp',
                to: to,
                type: 'text',
                text: { body: message }
            })
        }
    );

    if (!response.ok) return { success: false, reason: 'WhatsApp API error' };
    return { success: true };
}
```

10.2 Résultat de l'envoi

```
interface SendResult {
    success: boolean;
    reason?: string; // présent si success = false
}
```

Résultat	Cause
<code>{ success: true }</code>	Message envoyé avec succès
<code>{ success: false, reason: 'WhatsApp API error' }</code>	Réponse non-200 de Meta
<code>{ success: false, reason: 'Network error' }</code>	Erreur réseau (fetch échoué)
<code>{ success: false, reason: 'WhatsApp not configured' }</code>	Variables d'environnement manquantes

11. Templates WhatsApp Meta

11.1 Qu'est-ce qu'un template ?

Les **templates** sont des messages pré-approuvés par Meta. Ils sont obligatoires pour :

- ▶ Envoyer un message à un utilisateur qui n'a **pas interagi** dans les dernières 24h
- ▶ Envoyer des **notifications proactives** (confirmation commande, livraison, etc.)

11.2 Les 5 templates de Pièces

#	Nom	Catégorie	Usage
1	order_confirmation	UTILITY	Confirmation de commande
2	delivery_in_transit	UTILITY	Livraison en cours
3	delivery_completed	UTILITY	Livraison effectuée
4	owner_options	MARKETING	Options pour le propriétaire
5	artci_consent	UTILITY	Consentement données personnelles

11.3 Fonction d'envoi template

```
async function sendWhatsAppTemplate(
  to: string,
  templateName: string,
  params: string[]
): Promise<SendResult> {
  const response = await fetch(
    `https://graph.facebook.com/v18.0/${WHATSAPP_PHONE_ID}/messages`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${WHATSAPP_TOKEN}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        messaging_product: 'whatsapp',
        to: to,
        type: 'template',
        template: {
          name: templateName,
          language: { code: 'fr' },
          components: [{
            type: 'body',
            parameters: params.map(p => ({ type: 'text', text: p }))
          }]
        }
      })
    }
  );

  if (!response.ok) return { success: false, reason: 'WhatsApp API error' };
  return { success: true };
}
```

12. Template : order_confirmation

Détails

Propriété	Valeur
Nom	order_confirmation
Catégorie	UTILITY
Langue	fr
Paramètres	4

Corps du message

Commande confirmée 

Pièce : {{1}}

Vendeur : {{2}}

Prix total : {{3}} FCFA

Livraison estimée : {{4}}

Paramètres

#	Champ	Exemple
{{1}}	Nom de la pièce	Alternateur Toyota Corolla
{{2}}	Nom du vendeur	Auto Parts Abidjan
{{3}}	Prix total	45 000
{{4}}	Estimation livraison	J+2

Déclencheur

Envoyé quand la commande passe en statut **PAID** ou **VENDOR_CONFIRMED**.

Exemple rendu

Commande confirmée 

Pièce : Alternateur Toyota Corolla

Vendeur : Auto Parts Abidjan

Prix total : 45 000 FCFA

Livraison estimée : J+2

13. Template : delivery_in_transit

Détails

Propriété	Valeur
Nom	delivery_in_transit
Catégorie	UTILITY
Langue	fr
Paramètres	2

Corps du message

Votre pièce est en route 

Coursier `{{1}}` se dirige vers votre garage.

Arrivée estimée : `{{2}}`

Paramètres

#	Champ	Exemple
<code>{{1}}</code>	Nom du livreur	Konan Adèle
<code>{{2}}</code>	Heure d'arrivée estimée	14h30

Déclencheur

Envoyé quand la livraison passe en statut **IN_TRANSIT**.

14. Template : delivery_completed

Détails

Propriété	Valeur
Nom	delivery_completed
Catégorie	UTILITY
Langue	fr
Paramètres	0

Corps du message

Pièce livrée 

La pièce est-elle conforme ?

Répondez  ou 

Déclencheur

Envoyé quand la livraison passe en statut **DELIVERED**.

Interaction attendue

L'utilisateur peut répondre :

- ▶  → Confirmation de réception (à traiter par le bot)
- ▶  → Signalement de problème (à traiter par le bot)

Note v1 : La gestion des réponses  /  n'est pas encore implémentée dans le parseur de commandes.

15. Template : owner_options

Détails

Propriété	Valeur
Nom	owner_options
Catégorie	MARKETING
Langue	fr
Paramètres	8

Corps du message

Votre mécanicien {{1}} vous recommande :

1 {{2}} – {{3}} FCFA

2 {{4}} – {{5}} FCFA

3 {{6}} – {{7}} FCFA

Répondez 1, 2 ou 3 pour commander.

Ou consultez tous les détails : {{8}}

Paramètres

#	Champ	Exemple
{{1}}	Nom du mécanicien	Yao Kouadio
{{2}}	Pièce option 1	Alternateur neuf
{{3}}	Prix option 1	55 000
{{4}}	Pièce option 2	Alternateur occasion
{{5}}	Prix option 2	35 000
{{6}}	Pièce option 3	Alternateur reconditionné
{{7}}	Prix option 3	42 000
{{8}}	Lien vers la commande	pieces.ci/choose/abc123

Déclencheur

Envoyé par le mécanicien au propriétaire du véhicule avec les options de pièces recommandées.

Interaction attendue

L'utilisateur peut répondre **1, 2** ou **3** pour sélectionner une option, ou cliquer sur le lien pour voir plus de détails.

Note v1 : La gestion des réponses 1/2/3 n'est pas encore implémentée.

16. Template : artci_consent

Détails

Propriété	Valeur
Nom	artci_consent
Catégorie	UTILITY
Langue	fr
Paramètres	0

Corps du message

Bienvenue sur Pièces 

Avant de commencer, nous devons recueillir votre consentement pour le traitement de vos données personnelles (loi n°2013-450).

Données collectées : numéro de téléphone, historique commandes, localisation GPS (livraison).

Répondez OUI pour accepter.

Déclencheur

Envoyé au premier contact d'un nouvel utilisateur via WhatsApp, avant toute autre interaction.

Cadre légal

Conforme à la **Loi n°2013-450** relative à la protection des données à caractère personnel en Côte d'Ivoire (ARTCI).

Interaction attendue

L'utilisateur répond **OUI** pour donner son consentement.

Note v1 : La gestion de la réponse "OUI" pour enregistrer le consentement n'est pas encore implémentée dans le parseur.

17. Système de notifications

17.1 Architecture

Le module de notification est séparé du module WhatsApp :



17.2 Structure du payload

```
interface NotificationPayload {
  to: string; // Numéro de téléphone
  channel: 'whatsapp' | 'sms' | 'push';
  template?: string; // Nom du template WhatsApp
  message: string; // Texte du message (fallback)
  params?: string[]; // Paramètres du template
}
```

17.3 Envoi administrateur

Les administrateurs peuvent envoyer des notifications manuellement :

```
POST /api/v1/notifications/send
Auth: Bearer Token (rôle ADMIN requis)
Body: {
  "to": "+22507000000000",
  "channel": "whatsapp",
  "message": "Votre message personnalisé"
}
```

18. Notifications par événement

18.1 Fonctions de notification prédéfinies

Fonction	Événement	Message
<code>notifyOrderStatusChange</code>	Changement statut commande	Messages contextuels selon le nouveau statut
<code>notifyVendorNewOrder</code>	Nouvelle commande reçue	Alerte vendeur avec détails
<code>notifyVendorLowStock</code>	Stock critique	Alerte stock avec nom de l'article
<code>notifyRiderAssignment</code>	Livraison assignée	Détails du ramassage

18.2 Messages par statut de commande

Statut	Message envoyé
PAID	"Votre commande [ID:8] est confirmée. Le vendeur prépare votre pièce."
VENDOR_CONFIRMED	"Le vendeur a confirmé votre commande [ID:8]. Livraison en cours de préparation."
DISPATCHED	"Votre commande [ID:8] a été expédiée! Un livreur est en route."
DELIVERED	"Votre commande [ID:8] a été livrée. Confirmez la réception."
CANCELLED	"Votre commande [ID:8] a été annulée."

18.3 Notifications vendeur

Nouvelle commande :

"Nouvelle commande [ID:8]: [N] pièce(s). Confirmez dans les 45 minutes."

Stock critique :

"Stock critique: \"[nom article]\" est en rupture. Mettez à jour votre catalogue."

18.4 Notification livreur

Assignment :

"Nouvelle livraison [ID:8] assignée. Récupérez à: [adresse de ramassage]"

19. Architecture multi-canal

19.1 Canaux supportés

Canal	État	Implémentation
WhatsApp	Fonctionnel	Meta Cloud API v18.0
SMS	Stub	Retourne { success: false }
Push	Stub	Retourne { success: false } — PWA Service Worker prêt

19.2 Envoi multi-canal

```
async function sendMultiChannel(  
  channels: NotificationPayload[]  
) : Promise<SendResult[]> {  
  return Promise.allSettled(  
    channels.map(ch => sendNotification(ch))  
  );  
}
```

19.3 Exemple d'utilisation

```
// Envoyer sur WhatsApp ET SMS (quand SMS sera implémenté)
await sendMultiChannel([
  { to: phone, channel: 'whatsapp', message: 'Commande confirmée' },
  { to: phone, channel: 'sms', message: 'Commande confirmée' }
]);
```

20. Préférences de notification utilisateur

20.1 Modèle de données

```
model NotificationPreference {
  id          String   @id @default(cuid())
  userId      String   @unique
  user        User     @relation(fields: [userId])
  whatsapp    Boolean  @default(true)
  sms         Boolean  @default(false)
  push        Boolean  @default(false)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

20.2 Valeurs par défaut

Canal	Activé par défaut
WhatsApp	Oui
SMS	Non
Push	Non

20.3 Endpoints API

```
GET /api/v1/notifications/preferences
→ Auth requise
→ Retourne les préférences actuelles

PUT /api/v1/notifications/preferences
→ Auth requise
→ Body: { "whatsapp": true, "sms": false, "push": true }
```

20.4 Note d'implémentation

Les préférences sont stockées mais **pas encore vérifiées** avant l'envoi. L'intégration du contrôle opt-in/opt-out dans `sendNotification()` est planifiée.

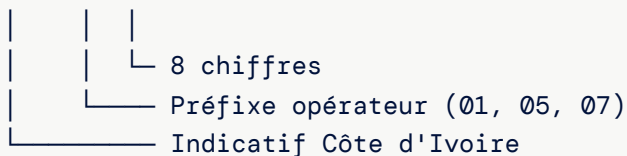
21. Format des numéros de téléphone

21.1 Validation Zod

```
const phoneSchema = z.string().regex(/^+225(01|05|07)\d{8}$/);
```

21.2 Format accepté

+225 XX XXXXXXXX



8 chiffres
Préfixe opérateur (01, 05, 07)
Indicatif Côte d'Ivoire

21.3 Opérateurs par préfixe

Préfixe	Opérateur
01	Orange CI
05	MTN CI
07	Moov Africa / Wave

21.4 Exemples

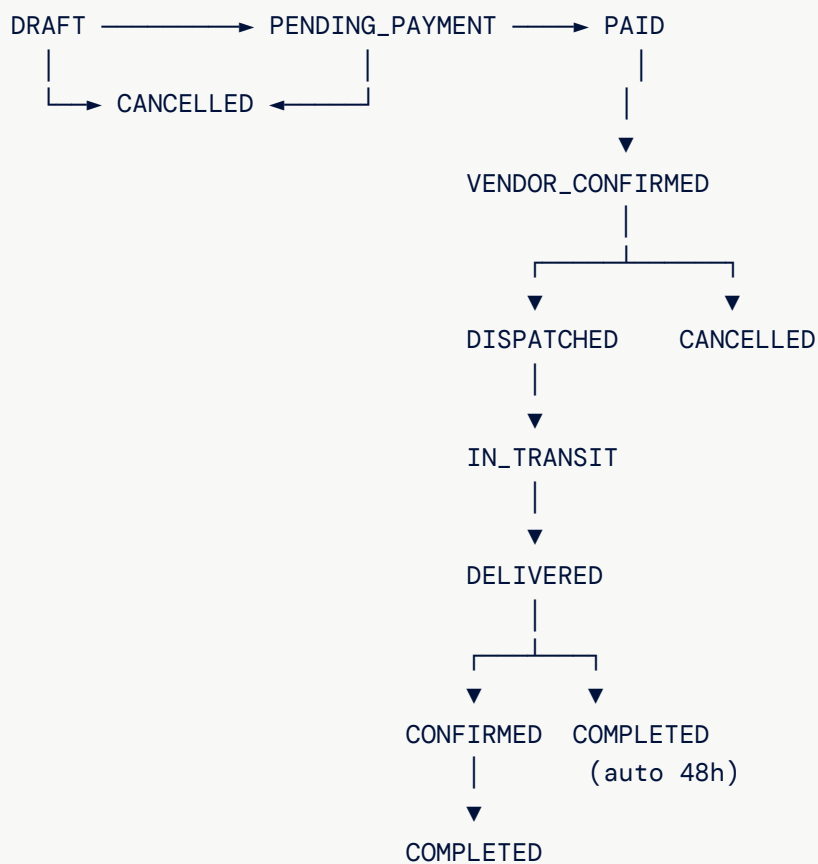
Numéro	Valide	Opérateur
+2250700000000	Oui	Moov/Wave
+2250100000000	Oui	Orange
+2250500000000	Oui	MTN
+2250300000000	Non	Préfixe invalide
0700000000	Non	Manque +225

21.5 Message d'erreur

"Numéro ivoirien invalide (format: +225 XX XX XX XX XX)"

22. Intégration avec la machine à états commande

22.1 Machine à états



22.2 Points d'intégration WhatsApp

Transition	Destinataire	Notification
→ PAID	Acheteur	Template <code>order_confirmation</code>
→ PAID	Vendeur	<code>notifyVendorNewOrder()</code>
→ VENDOR_CONFIRMED	Acheteur	Texte confirmation vendeur
→ DISPATCHED	Acheteur	Texte expédition
→ IN_TRANSIT	Acheteur	Template <code>delivery_in_transit</code>
→ DELIVERED	Acheteur	Template <code>delivery_completed</code>
→ CANCELLED	Acheteur	Texte annulation

22.3 Événements enregistrés

Chaque transition est enregistrée dans la table `OrderEvent` :

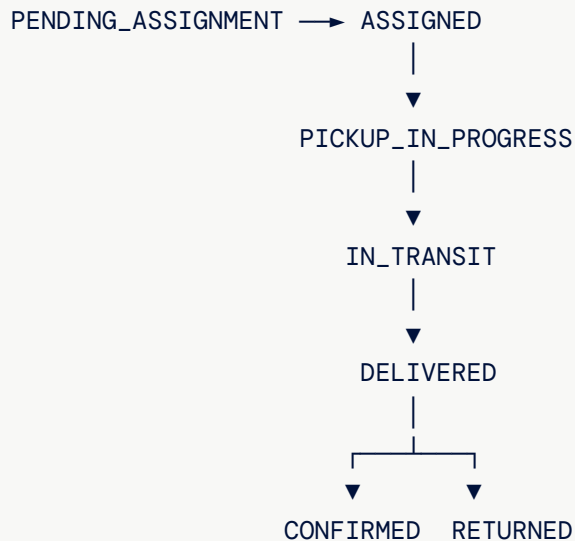
```
model OrderEvent {
  id          String      @id @default(cuid())
  orderId     String
  order       Order       @relation(fields: [orderId])
  fromStatus  OrderStatus
  toStatus    OrderStatus
  actorId     String?
  note        String?
  createdAt   DateTime    @default(now())
}
```

22.4 Note d'implémentation

La fonction `transitionOrder()` met à jour le statut et crée l'événement, mais **n'appelle pas encore les fonctions de notification**. Le câblage est planifié.

23. Intégration avec la machine à états livraison

23.1 Machine à états



23.2 Points d'intégration WhatsApp

Transition	Destinataire	Notification
→ ASSIGNED	Livreur	<code>notifyRiderAssignment()</code>
→ IN_TRANSIT	Acheteur	Template <code>delivery_in_transit</code>
→ DELIVERED	Acheteur	Template <code>delivery_completed</code>
Client absent	Acheteur	Notification client absent

23.3 Cas spécial : client absent

Quand le livreur marque **"Client absent"** :

- ▶ Le champ `clientAbsent` passe à `true` dans la table Delivery
- ▶ Une notification devrait être envoyée à l'acheteur
- ▶ Une nouvelle tentative de livraison est organisée

24. Rate limiting et gestion d'erreurs

24.1 Rate limiting API

Scope	Limite	Plugin
Global (toutes routes)	100 req/min par IP	@fastify/rate-limit
Auth OTP	5 req/min par IP	Spécifique
WhatsApp webhook	Non limité côté serveur	Meta gère en amont

24.2 Limite Meta Cloud API

Élément	Limite
Messages/seconde	80 par compte business
Conversations/jour	Variable selon le tier du compte

24.3 Gestion d'erreurs

Erreur	Comportement
HMAC invalide	→ 401 Unauthorized
Payload mal formé	→ 200 OK (acquiescement)
API Meta non disponible	→ { success: false, reason: 'Network error' }
API Meta erreur 4xx/5xx	→ { success: false, reason: 'WhatsApp API error' }
Variables non configurées	→ { success: false, reason: 'WhatsApp not configured' }

24.4 Pas de mécanisme de retry

L'implémentation actuelle **ne retente pas** les envois échoués.

Planifié : File d'attente asynchrone (`pgqueue`) pour les jobs `NOTIFICATION.SEND_WHATSAPP` avec logique de retry et backoff exponentiel.

25. Tests unitaires

25.1 Fichier de tests

`apps/api/src/modules/whatsapp/whatsapp.routes.test.ts`

25.2 Cas de test

Test	Description
Vérification webhook GET	Vérifie la réponse challenge avec le bon verify_token
Rejet webhook GET	Vérifie le 403 avec un mauvais verify_token
Signature HMAC valide	Vérifie le traitement du message avec signature correcte
Signature HMAC invalide	Vérifie le 401 avec une signature incorrecte
Commande "aide"	Vérifie la réponse d'aide
Commande "recherche"	Vérifie la réponse de recherche
Image reçue	Vérifie l'accusé de réception photo
Commande inconnue	Vérifie le message d'erreur

25.3 Mocking

Les tests mockent :

- ▶ `sendWhatsAppMessage()` pour éviter les appels réels à Meta
- ▶ La vérification HMAC pour isoler les tests de logique métier
- ▶ Le `fetch()` global pour simuler les réponses API

26. État de l'implémentation

26.1 Ce qui est implémenté (v1)

Composant	État	Fichier
Webhook GET (vérification Meta)	Implémenté	<code>whatsapp.routes.ts</code>
Webhook POST (réception messages)	Implémenté	<code>whatsapp.routes.ts</code>
Sécurité HMAC SHA-256	Implémenté	<code>whatsapp.service.ts</code>
Parse des messages entrants	Implémenté	<code>whatsapp.service.ts</code>
Commandes bot (aide, recherche, image)	Implémenté	<code>whatsapp.service.ts</code>
Envoi de messages texte	Implémenté	<code>whatsapp.service.ts</code>
Envoi de templates	Implémenté	<code>whatsapp.service.ts</code>
Service de notification	Implémenté	<code>notification.service.ts</code>
Préférences de notification	Implémenté	<code>notification.routes.ts</code>
Templates documentés	Implémenté	<code>whatsapp-templates.md</code>
Tests unitaires	Implémentés	<code>whatsapp.routes.test.ts</code>

26.2 Ce qui n'est PAS encore câblé

Composant	État	Action requise
Notifications sur transitions commande	Non câblé	Appeler <code>notifyOrderStatusChange()</code> dans <code>order.service.ts</code>
Notifications sur transitions livraison	Non câblé	Appeler <code>notifyRiderAssignment()</code> dans <code>delivery.service.ts</code>
Vérification préférences avant envoi	Non câblé	Vérifier opt-in dans <code>sendNotification()</code>
Téléchargement images pour identification IA	Non câblé	Intégrer Media API + Gemini Vision
Gestion réponses  /  (<code>delivery_completed</code>)	Non câblé	Ajouter au parseur de commandes
Gestion réponses 1/2/3 (<code>owner_options</code>)	Non câblé	Ajouter au parseur de commandes
Gestion réponse OUI (<code>artci_consent</code>)	Non câblé	Enregistrer consentement en DB
File d'attente async (pgqueue)	Non câblé	Implémenter worker de notification
Canal SMS	Stub	Intégrer Twilio ou Orange SMS API
Canal Push	Stub	Intégrer Web Push via Service Worker

27. Évolutions planifiées

27.1 Phase 2 : Conversations multi-tours

MÉCANICIEN (WhatsApp)

BOT PIÈCES

"Je cherche un alternateur
pour Toyota Corolla 2017"

→ "J'ai trouvé 3 options :
1. Neuf – 55 000 FCFA
2. Occasion – 35 000 FCFA
3. Reconditionné – 42 000 FCFA
Répondez 1, 2 ou 3"

"2"

→ "Alternateur occasion sélectionné.
Créer la commande pour votre client ?
Répondez OUI ou NON"

"OUI"

→ "Commande créée! Envoyez ce lien
à votre client pour le paiement :
pieces.ci/choose/abc123xyz"

27.2 Phase 2 : File d'attente asynchrone

Événement (ex: commande payée)



Ajout job dans pgqueue :

```
type: NOTIFICATION.SEND_WHATSAPP  
payload: { to, template, params }
```



Worker pgqueue traite le job :

- sendWhatsAppTemplate()
- Si échec : retry avec backoff
- Max 3 retries
- Log du résultat

27.3 Phase 2 : Messages interactifs

```
{
  "type": "interactive",
  "interactive": {
    "type": "button",
    "body": { "text": "Votre commande est confirmée" },
    "action": {
      "buttons": [
        { "type": "reply", "reply": { "id": "track", "title": "Suivre" } },
        { "type": "reply", "reply": { "id": "cancel", "title": "Annuler" } }
      ]
    }
  }
}
```

27.4 Phase 2 : Suivi livraison en direct

```
Livreur met à jour position GPS
    |
    ▼
Calcul ETA dynamique
    |
    ▼
WhatsApp push : "Votre livreur est à 5 min"
    |
    ▼
Alerte SLA : si retard > seuil
    |
    ▼
WhatsApp proactif : "Retard constaté –
Votre prochaine livraison Standard offerte"
```

28. Processus de soumission des templates Meta

28.1 Étapes

1. Accédez au **WhatsApp Business Manager** (business.facebook.com)
2. Naviguez vers **Message Templates**
3. Cliquez sur **Create Template**
4. Remplissez : nom, catégorie, langue, corps du message
5. Soumettez pour approbation

28.2 Délais

Étape	Délai
Soumission → Première réponse	24-48 heures
Rejet → Re-soumission	Immédiat
Nombre d'itérations prévu	2-3 avant approbation
Planning	Soumettre dès Story 1.1, avant Epic 6

28.3 Raisons de rejet courantes

Raison	Solution
Contenu promotionnel dans catégorie UTILITY	Reclassifier en MARKETING
Paramètres mal formatés	Vérifier <code>{{1}}</code> , <code>{{2}}</code> , etc.
Langue non spécifiée	Ajouter <code>fr</code> comme langue
Contenu ambigu	Clarifier le texte du message
URL dans le corps	Utiliser un bouton CTA à la place

29. Référence des fichiers

29.1 Module WhatsApp

Fichier	Chemin	Rôle
Routes	<code>apps/api/src/modules/whatsapp/whatsapp.routes.ts</code>	Endpoints webhook GET et POST
Service	<code>apps/api/src/modules/whatsapp/whatsapp.service.ts</code>	Logique métier : parse, envoi, HMAC
Tests	<code>apps/api/src/modules/whatsapp/whatsapp.routes.test.ts</code>	Tests unitaires webhook

29.2 Module Notification

Fichier	Chemin	Rôle
Service	<code>apps/api/src/modules/notification/notification.service.ts</code>	Fonctions de notification
Routes	<code>apps/api/src/modules/notification/notification.routes.ts</code>	Endpoints préférences + envoi admin
Validators	<code>packages/shared/validators/notification.ts</code>	Schémas Zod de validation

29.3 Configuration

Fichier	Chemin	Rôle
Templates	<code>docs/whatsapp-templates.md</code>	Documentation des 5 templates Meta
Auth validator	<code>packages/shared/validators/auth.ts</code>	Validation format téléphone +225
Prisma schema	<code>packages/shared/prisma/schema.prisma</code>	Modèle NotificationPreference
Rate limit	<code>apps/api/src/plugins/rateLimit.ts</code>	Configuration rate limiting
Server	<code>apps/api/src/server.ts</code>	Enregistrement des modules

30. FAQ technique

Q : Le bot peut-il fonctionner sans les variables WhatsApp configurées ?

R : Oui. Le bot démarre sans erreur. Les messages entrants sont traités mais les réponses ne sont pas envoyées(`{ success: false, reason: 'WhatsApp not configured' }`). C'est une dégradation gracieuse.

Q : Les templates doivent-ils être approuvés avant le test ?

R : Non. Pour le développement, vous pouvez envoyer des messages texte simples (pas de templates) aux numéros enregistrés dans votre sandbox Meta. Les templates sont nécessaires pour les messages proactifs en production.

Q : Comment tester le webhook localement ?

R : Utilisez un tunnel (ngrok, Cloudflare Tunnel) pour exposer votre serveur local :

```
ngrok http 3001
```

Puis configurez l'URL du webhook dans le WhatsApp Business Manager :

```
https://xxx.ngrok.io/api/v1/whatsapp/webhook
```

Q : Que se passe-t-il si Meta renvoie le même message ?

R : Le serveur retourne toujours 200 OK, même en cas d'erreur de traitement. Cela empêche Meta de renvoyer le message. Le bot est idempotent — traiter le même message deux fois produit le même résultat.

Q : Le bot gère-t-il les conversations de groupe ?

R : Non. Le bot traite uniquement les messages individuels (1:1). Les messages de groupe WhatsApp ne sont pas supportés par la Meta Cloud API pour les comptes business.

Q : Quelle est la différence entre messages UTILITY et MARKETING ?

R : Les templates **UTILITY** concernent les transactions (confirmations, livraisons). Les templates **MARKETING** concernent les recommandations et offres. Meta applique des tarifs différents et des règles d'opt-in plus strictes pour MARKETING.

Q : Comment ajouter une nouvelle commande au bot ?

R : Modifiez la logique de routage dans `whatsapp.service.ts` . Ajoutez un nouveau cas dans la chaîne if/else du parseur de commandes. Pas de configuration externe nécessaire.

Q : Le bot supporte-t-il les réponses rapides (quick replies) ?

R : Pas encore en v1. Les messages interactifs WhatsApp (boutons, listes) sont prévus pour la Phase 2. Actuellement, le bot attend des réponses textuelles simples.

Q : Comment monitorer les envois WhatsApp ?

R : En v1, les erreurs d'envoi sont logguées dans la sortie standard du serveur Fastify. Un dashboard de monitoring est prévu avec l'implémentation de la file d'attente pgqueue en Phase 2.

Q : Le bot peut-il envoyer des médias (images, documents) ?

R : L'infrastructure Meta Cloud API le permet, mais ce n'est pas encore implémenté dans Pièces v1. L'envoi d'images (résultats d'identification IA) est prévu pour la Phase 2.